Operating Systems Lab Manual F2025

Lab 7 and 8: A Simple Dropbox Clone

Semester: Fall 2025

Project Guidlines

- Form group of 3 and share your group details in the sheet shared.
- Each group must track their project remotely on github or any git repository hosting platform and share link to the public repository.
- The **deadline** for phase 1 is till lab 8. For phase 2, it will be announced later.

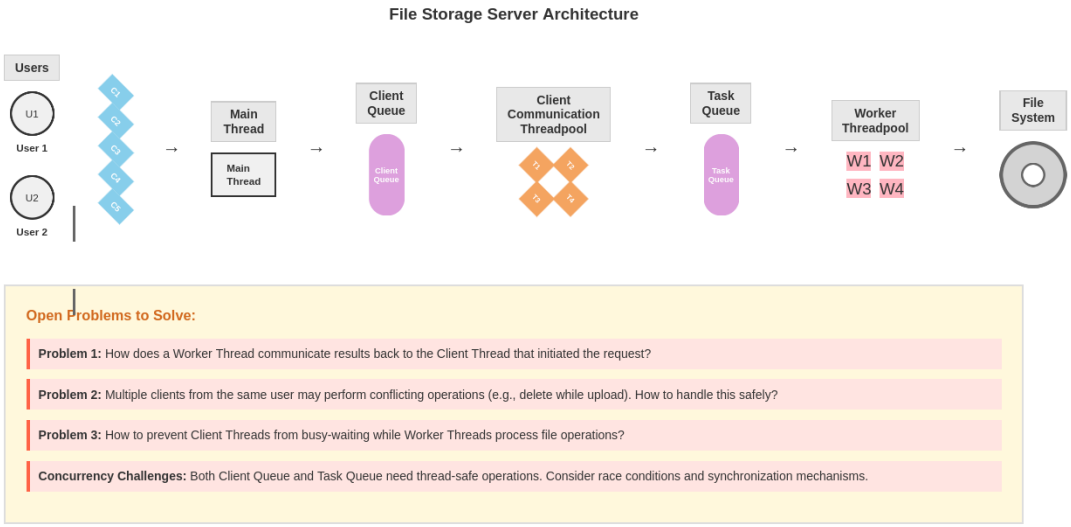
Objective

This lab project tasks students with designing and implementing a multi-threaded file-storage server that offers core file operations (UPLOAD, DOWNLOAD, DELETE, LIST) and user management (signup/login, per-user storage quotas). The project is intended to teach and assess practical understanding of the following concepts:

- Design and implementation of producer/consumer queues and thread pools.
- Thread synchronization primitives (mutexes, condition variables) and avoidance of race conditions.
- Coordination between different thread roles.
- Correct resource management and debugging techniques.
- Use of tools to detect memory leaks and concurrency issues (Valgrind and Thread-Sanitizer).
- Design decisions for handling concurrent client interactions for the same user and conflict resolution.

At the end of the project students must deliver a working server implementation, a short design write-up describing synchronization/communication choices and trade-offs, and automated test steps that demonstrate correctness under concurrent load. Grading will focus both on correctness (functional tests) and on absence of low-level bugs (memory leaks, data races).

Server Architecture



Server Architecture description

The server is built in three layers. The main thread accepts client connections and pushes them into a Client Queue. A Client Threadpool takes sockets from this queue, handles login/signup, and receives commands from clients. Instead of executing commands directly, client threads put them into a global Task Queue.

A Worker Threadpool takes tasks from this queue and performs the actual operations such as file upload, download, delete, or list. Since multiple clients can belong to the same user, workers must ensure conflicting actions are handled safely.

An open design question is how workers send results back to the correct client thread, since only client threads are allowed to communicate with sockets. Students must propose a solution to this.

Both queues are shared structures, so proper synchronization is required to avoid race conditions.

Detailed Project Description

This project implements a simplified Dropbox-like file storage server that accepts concurrent client connections and supports the following commands from authenticated users: `UPLOAD <filename>`, `DOWNLOAD <filename>`, `DELETE <filename>`, and `LIST`.

High-level architecture. The server is organized into three logical layers:

1. **Accept/Main thread.** A single main thread listens for incoming TCP connections. For each accepted connection, it pushes the associated socket descriptor into a thread-safe *Client Queue*.
2. **Client Threadpool.** A pool of threads consumes the Client Queue. Each client thread takes ownership of one socket at a time, performs authentication (signup/login), and then receives textual commands from that client. When a client issues an operation (e.g., `UPLOAD`), the client thread packages the request into a *Task* structure and pushes it into the global *Task Queue*. Client threads will busy wait until their task is completed. (***Check Bonus Tasks***).
3. **Worker Threadpool.** A separate pool of worker threads consumes the Task Queue. Workers perform the heavy lifting: file I/O (read/write/delete), quota checking, and metadata updates to the server's storage structures(if any)

Key constraints and correctness requirements.

- **Concurrency safety:** Both queues and all shared data structures must be protected from data races. Use mutexes and condition variables (or higher-level concurrency constructs). Designs will be validated using ThreadSanitizer.
- **Memory safety:** No memory leaks or use-after-free bugs. All allocated resources (memory, file descriptors, threads) must be released at shutdown. Validated using Valgrind or equivalent.
- **Multi-client per user:** The server must correctly handle a single user with multiple simultaneous client sessions. In particular, conflicting operations on the same file (e.g., concurrent delete and upload) must be handled so data remains consistent.

Open design problem (to be discussed and justified). A deliberate open design decision is how **workers deliver results back to the correct client session**. Students must choose and justify one approach; Students must explain in their design write-up which option they chose and why (trade-offs in complexity, fairness, latency, and resource usage).

Outcomes

1. Build a multi-threaded server that supports user registration, login, and per-user storage quotas.
2. Build a simple client which can communicate with this server
3. Implement the following file operations:
   (a) `UPLOAD <filename>`
   (b) `DOWNLOAD <filename>`
   (c) `DELETE <filename>`
   (d) `LIST` - lists all user files
4. Design and use two thread-safe queues
   - A Client Queue (for new connections)
   - A Task Queue (for user commands)
5. Implement and manage two thread pools.
   - Client Threadpool (for communication)
   - Worker Threadpool (for execution)
6. Handle concurrent access to shared resources, ensuring correctness even if multiple clients for the same user issue conflicting commands.
7. Propose and implement a worker-to-client communication mechanism for sending task results back.
8. Produce a short design report explaining synchronization decisions and the chosen worker→client communication method.

Requirements

1. Makefile or a script to compile the server with a single `make` command
2. No memory leaks (checked with Valgrind)
3. No race conditions (checked with ThreadSanitizer)
4. Proper synchronization (mutexes, condition variables, or equivalent)

Bonus

- No busy waiting in client threads.
- Implement encoding and decoding in the `UPLOAD` and `DOWNLOAD` operations
- Implement a priority system where some tasks/users are given priority over others.

Reading Material

- **Threadpool in C** : `https://nachtimwald.com/2019/04/12/thread-pool-in-c/`
- **Multi Threading in C** : `https://man7.org/linux/man-pages/man7/pthreads.7.html`
- **Network Programming in C** : `https://beej.us/guide/bgnet/` Check out chapter 6 in particular

Phase 1: Core architecture and single-session functionality

Goal

Build the server skeleton and prove the producer/consumer architecture works end-to-end for a ***single client session per user***.

Required tasks

1. Implement the main accept loop that pushes accepted socket descriptors into a Client Queue
2. Implement a Client Threadpool that dequeues sockets, performs signup/login, and parses textual commands. Client threads must not execute heavy operations directly.
3. Implement a global Task Queue and a Worker Threadpool that can dequeue tasks and execute them
4. Implement the four basic commands with correct I/O to disk for a single client per user: `UPLOAD DOWNLOAD DELETE LIST`
5. Simple metadata structure mapping user → files and quota accounting.
6. Synchronization for the two queues
7. Provide a Makefile, build instructions, and a README describing how to run a single client test
8. Provide minimal tests that demonstrate one client can: sign up/login, upload, list, download, and delete files.

Acceptance criteria

- Functional test: a single client session can perform all four operations successfully.
- Basic Valgrind run shows no obvious leaks for the scenario exercised.
- Code builds with make and README reproduces steps to run

Deliverables

- Source code + Makefile.
- Short Phase-1 report (1 page at max) describing threadpool and queue design and synchronization choices.

Phase 2: Concurrency, correctness, robustness and final touches

Goal

Extend Phase 1 to correctly handle multiple simultaneous clients ***(including multiple sessions per same user)***, implement worker→client result delivery, harden synchronization, add testing for races/leaks, and optionally add selected bonus features.

Required Tasks

1. Support **multiple concurrent clients**, including multiple simultaneous sessions for the same user
2. Implement a worker→client result delivery design (choose and justify one)
3. Implement per-user / per-file concurrency control so conflicting operations serialize safely. Document and justify the choice.
4. Improve metadata management and atomic updates (ensure consistency when multiple workers update metadata).
5. Add robust shutdown handling: close sockets, signal threads to exit, free memory.
6. Full testing: concurrency tests that spawn many clients doing varied operations, and automated Valgrind + ThreadSanitizer runs.
7. Fix any race conditions and leaks uncovered by TSAN/Valgrind.
8. Complete design report (1–2 pages) explaining worker→client mechanism, per-user locking, and why the approach is race-free.

Acceptance criteria

1. Correct behavior under concurrent load: automated tests with multiple clients must pass (no corrupted metadata, operations succeed or fail with proper errors, consistent state).
2. ThreadSanitizer reports no data races on the exercised test suite.
3. Valgrind shows no memory leaks for representative concurrent tests.
4. Worker→client communication must reliably deliver responses to the correct socket/session.
5. Clean shutdown with no resource leaks.

Deliverables

- Updated source + Makefile + concurrency test scripts
- Phase-2 design report (1–2 pages) describing concurrency controls and worker→client mechanism, plus trade-offs and limitations.
- A short demo script that runs the concurrency tests and prints TSAN/Valgrind summaries (or instructions to run them).